

INSTITUTO FEDERAL
Minas Gerais
Campus Bambuí

Instituto Federal de Minas Gerais
Curso de Engenharia de Computação
Professora Patrícia Aparecida Proença Avila

RELATÓRIO: SELEÇÃO DE ATIVIDADES

Alunos: Paulo Henrique Ferreira e
Maria Eduarda Saldanha Alves

Bambuí - MG
2026

Conteúdo

1 Objetivo **1**

2 Representação dos dados **2**

3 Leitura dos arquivos **3**

4 Ordenação pelo horário de término **4**

5 Estratégia gulosa **5**

6 Teste realizado **6**

7 Contraexemplo — ordenação pelo início **7**

8 Conclusão **8**

1 Objetivo

O objetivo dessa atividade é implementar o problema da Seleção de Atividades utilizando uma estratégia gulosa. O problema consiste em selecionar a maior quantidade possível de atividades que não possuam sobreposição de horários. Para isso, cada atividade possui um identificador, um horário de início e um horário de término. Os dados são armazenados em um arquivo de texto, lidos pelo programa e posteriormente processados para encontrar uma solução.

2 Representação dos dados

Para representar cada atividade, foi utilizada uma estrutura (`struct`) contendo três informações: o identificador da atividade, seu horário de início e seu horário de término. Após a definição da estrutura, foi criado um vetor de atividades para armazenar os dados lidos do arquivo.

```
1 typedef struct {
2     int id, inicio, fim;
3 } Atividades;
```

Dessa forma, cada posição do vetor representa uma atividade completa, contendo suas respectivas informações.

3 Leitura dos arquivos

Os dados das atividades são armazenados no arquivo `dados.txt`. O programa utiliza a função `fopen()` para abrir o arquivo no modo leitura. Em seguida, a função `fscan()` é utilizada para ler o identificador, o horário de início e o horário de término de cada atividade. Os dados são lidos e armazenados no vetor de atividades. Ao finalizar a leitura, o arquivo é fechado utilizando `fclose()`.

```
1     while(fscanf(arquivo, "%d %d %d",
2         &atividades[n].id,
3         &atividades[n].inicio,
4         &atividades[n].fim) == 3)
5     {
6             n++;
7     }
```

O `while` continua enquanto conseguir ler os três valores de uma atividade.

4 Ordenação pelo horário de término

Após a leitura dos dados, as atividades são ordenadas de acordo com o horário de término, em ordem crescente. Para realizar essa ordenação, foi utilizada a função `qsort()`, disponível na biblioteca `stdlib.h`. Como `qsort()` precisa de uma funçã0 que determine o critério de comparação, foi criada a função `compararFim()`.

A função `compararFim()` compara o horário de término de duas atividades. Dessa forma, as atividades que possuem menor horário de término são posicionadas primeiro no vetor.

```
1 int compararFim(const void *a, const void *b) {
2     Atividades *atividadeA = (Atividades *)a;
3     Atividades *atividadeB = (Atividades *)b;
4 
5     return atividadeA -> fim - atividadeB -> fim;
6 }
```

5 Estratégia gulosa

Após ordenar as atividades pelo horário de término, é aplicada a estratégia gulosa. A primeira atividade compatível é selecionada e, em seguida, o programa percorre as demais atividades procurando aquelas que possam ser realizadas sem sobreposição.

A condição utilizada é essa:

```
1 if (atividades[i].inicio >= ultimoFim)
```

Uma atividade é selecionada quando seu horário de início é maior ou igual ao horário de término da última atividade selecionada. Quando uma atividade é escolhida, a variável `ultimoFim` é atualizada para o horário de término dessa atividade. Esse processo continua até que todas as atividades sejam analisadas.

6 Teste realizado

Exemplo colocado no arquivo `dados.txt`:

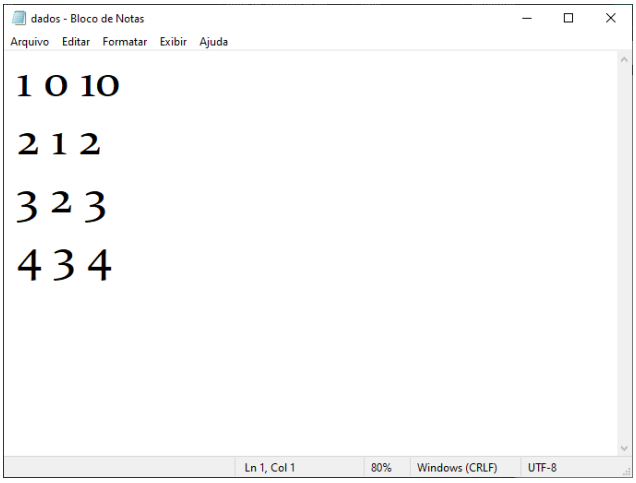


Figura 1: Exemplo de atividades colocadas no arquivo `dados.txt`

Para testar o funcionamento do algoritmo, foi utilizado um conjunto de quatro atividades. Após a ordenação pelo horário de término, a ordem de análise passa a ser:

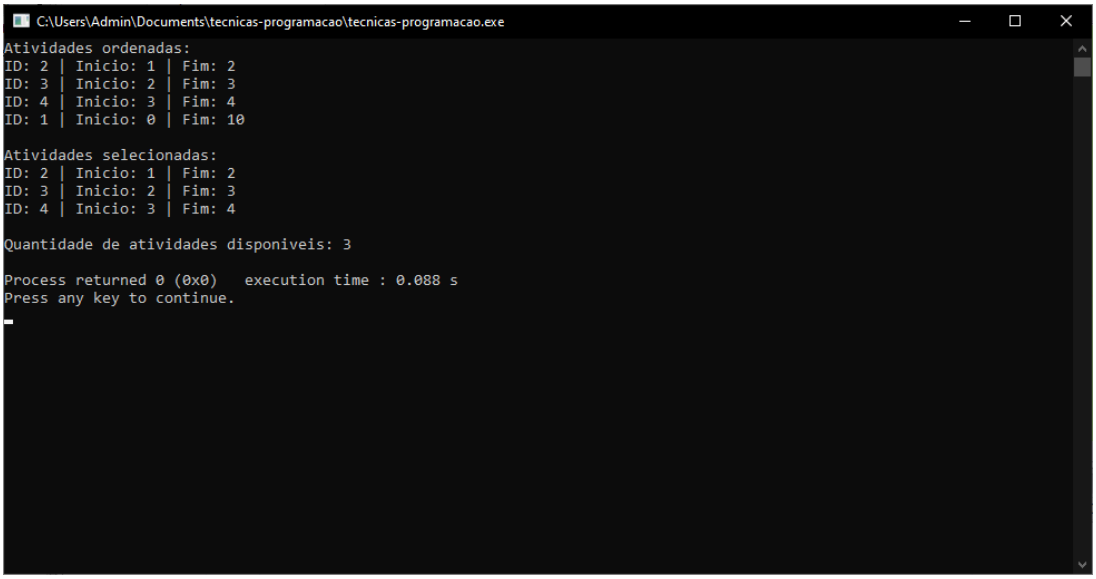


Figura 2: Algoritmo executado

O algoritmo seleciona as atividades 2,3 e 4, pois elas não possuem sobreposição de horários. Portanto, foram selecionadas 3 atividades.

7 Contraexemplo — ordenação pelo início

Para demonstrar que a ordenação pelo horário de início não garante uma solução ótima, pode-se utilizar o mesmo conjunto de atividades:

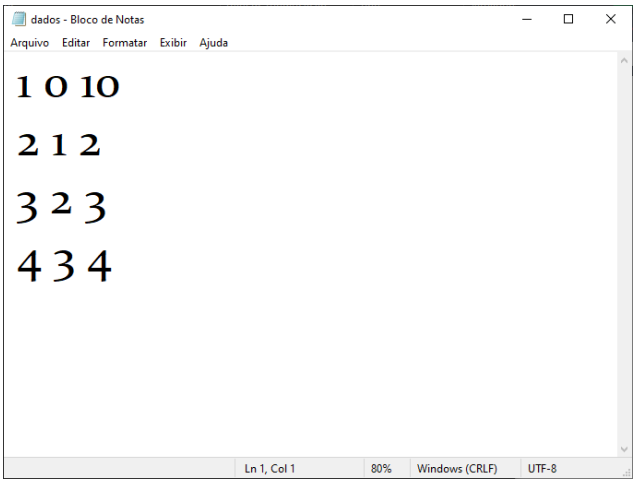


Figura 3: Exemplo de atividades colocadas no arquivo `dados.txt`

Se as atividades fossem ordenadas pelo horário de início, a atividade 1 seria analisada primeiro. Como ela começa em 0 e termina em 10, sua escolha impediria a seleção das demais atividades. Dessa forma, seriam selecionadas apenas 1 atividade.

Esse exemplo mostra que escolher a atividade que começa primeiro pode produzir uma solução pior. Já a escolha baseada no menor horário de término permite aproveitar melhor o tempo disponível.

8 Conclusão

A implementação permitiu aplicar o algoritmo guloso ao problema de Seleção de Atividades. Os dados foram lidos de um arquivo de texto, armazenados em um vetor e ordenados pelo horário de término. Em seguida, foram selecionadas as atividades compatíveis, buscando obter a maior quantidade possível de atividades sem sobreposição. O contraexemplo também demonstrou que ordenar pelo horário de início pode produzir uma solução inferior, justificando a utilização do horário de término como critério da estratégia gulosa.